

LLM Fundamentals

Module 6 — AI Engineering · Notebook 21

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

Contents

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

Outline

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

What is a Large Language Model?

A next-token predictor

An LLM learns statistical relationships between words from massive corpora. Given some text, it computes the *most probable continuation*, one token at a time.

Analogy

Someone who has read every book ever written: they can continue any topic plausibly — but have *no built-in sense* of what is true, current, or private to your company.

How LLMs see text: tokens

ChatGPT  = 3 tokens

The cat sat.  = 4 tokens

- Cost and the **context window** are measured in tokens — not characters, not words.
- Rule of thumb: 1 English token $\approx$ 4 characters $\approx$ 0.75 words (1,000 tokens $\approx$ 750 words).
- Code and non-English text tokenize *less* efficiently; the model is blind to characters *within* a token (“how many r’s in strawberry?”).

What are parameters? Why “large”?

Model	Parameters	Year
GPT-2	1.5 billion	2019
Llama-3 (open-weights)	8B / 70B / 405B	2024
Frontier (GPT-5.x, Claude, Gemini)	hundreds of billions (est.)	2023+

Key insight

There is *no* manual list of facts inside the model. Knowledge is *implicitly distributed* across all the weights — which is exactly why it can be confidently wrong.

Outline

- 1 Tokens
- 2 Training**
- 3 Transformer
- 4 Using LLMs
- 5 Limits

The training objective: next-token prediction

Hide → predict → measure → nudge, trillions of times

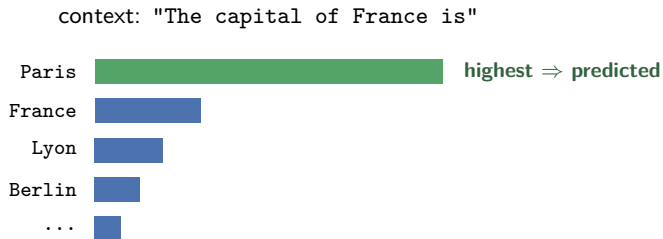
Hide the next token, ask the model to predict it, measure the error, nudge every parameter slightly toward a better prediction.

$$\mathcal{L} = - \sum_i y_i \log \hat{p}_i$$

where $y_i = 1$ for the one true next token and 0 otherwise (a *one-hot* target), and $\hat{p}_i$ is the model's predicted probability for token i . Grammar, facts, and reasoning emerge as *side-effects*.

You write this exact loop — loss → backward → step — yourself in **Module 17 (PyTorch)**; an LLM trains with the same five moves, just with more parameters and more data.

The output is a probability over *every* token



The model never returns “the answer” — it returns a *distribution*. temperature controls how often it picks something other than the top token.

Outline

- 1 Tokens
- 2 Training
- 3 Transformer**
- 4 Using LLMs
- 5 Limits

The architecture: the Transformer

A stack of attention + feed-forward layers

Each token becomes a vector; each layer lets tokens *exchange information* (attention), then transforms each vector (feed-forward). Stack many layers → rich, context-aware representations.

- All tokens are processed *in parallel* during training — a huge speed-up over older recurrent models.
- Depth (layers) + width (vector size) + data = the “large” in LLM.

The Attention mechanism (Q / K / V)

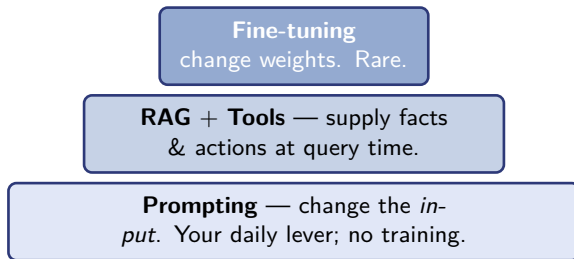
Step	What happens
1. Project	compute Query, Key, Value for every token via learned matrices
2. Score	$\text{score}_{ij} = Q_i \cdot K_j / \sqrt{d_k}$
3. Normalise	softmax over $j \rightarrow$ weights summing to 1
4. Mix	new vector = $\sum_j \text{weight}_{ij} V_j$

d_k is the length of each Key/Query vector; dividing by $\sqrt{d_k}$ keeps scores from blowing up. **Intuition:** for each word, look at every other word and decide how much to listen to it.

Outline

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

Three levels of using an LLM



Fine-tuning vs RAG

Fine-tuning changes *behaviour* / *style*. **RAG** (NB 23) keeps the weights frozen and supplies *fresh facts* — don't fine-tune to add knowledge.

Five prompting techniques

Technique	When to reach for it
Zero-shot	a general question; you trust the defaults
Few-shot	you want a <i>consistent</i> format — show 2–3 examples
Chain-of-Thought	multi-step reasoning — ask it to work step by step
Role assignment	set tone / depth — “you are a senior analyst. . .”
Format constraint	machine-parseable output — “return JSON with keys. . .”

A production prompt stacks several: *role* + *context* + *task* + *format* + *constraints*.

Outline

- 1 Tokens
- 2 Training
- 3 Transformer
- 4 Using LLMs
- 5 Limits

Two structural limitations

1. Hallucinations

The model predicts *plausible* continuations, not *true* ones — so it can invent clauses, statistics, or APIs with full confidence.

2. Knowledge cutoff

It knows nothing after its training date, and nothing about your *private* data — no current prices, no internal reports.

Mitigations (built later in the course)

RAG (NB 23) grounds answers in evidence · **Tools** (NB 24) let it look things up · **Evaluation** (NB 26) measures how often it's wrong.